Motif Finding (2)

BioInf 2025/2026 1st Semester

André Lamúrias

a.lamurias@fct.unl.pt

Recap

- Motif finding: comparing DNA sequences to find common subsequence
- Consensus, profile matrix, median string problem, branch and bound, heuristic consensus
- Now we are going to use probabilities

Most probable k-mer

- We can calculate the probability of this motif on any sequence

Profile

A:	.2	.2	.0	.0	.0	.0	.9	.1	.1	.1	.3	.0
C:	.1	.6	.0	.0	.0	.0	.0	.4	.1	.2	.4	.6
G:	.0	.0	1	1	.9	.9	.1	.0	.0	.0	.0	.0
T:	.7	.2	.0	.0	.1	.1	.0	.5	.8	.7	.3	.4

$$\Pr(\mathtt{ACGGGGATTACC}|\mathit{Profile}) = .2 \cdot .6 \cdot 1 \cdot 1 \cdot .9 \cdot .9 \cdot .9 \cdot .5 \cdot .8 \cdot .1 \cdot .4 \cdot .6 = 0.000839808$$

- We can test every possible k-mer on a each sequence to find the one most probable (BA2C)
- Example: ggtACGGGGATTACCt
  - ACGGGGATTACC is the most probable 12-kmer
  - What is the probability of the other 12-kmers?

Greedy Motif Search (BA2D)

```
GREEDYMOTIFSEARCH(Dna, k, t)
    BestMotifs ← motif matrix formed by first k-mers in each string from Dna
    for each k-mer Motif in the first string from Dna
        Motif_1 ← Motif
        for i = 2 to t
            form Profile from motifs Motif_1, ..., Motif_{i−1}
            Motif_i ← Profile-most probable k-mer in the i-th string in Dna
        Motifs ← (Motif_1, ..., Motif_t)
        if SCORE(Motifs) < SCORE(BestMotifs)
            BestMotifs ← Motifs
    return BestMotifs
```

Greedy Motif Search (BA2D)

- What are the advantages and disadvantages of this approach?
  - Fast but…
  - Does not achieve the best solution – why?
- Make the profile matrix assuming we start with the upper case kmer:
  - What is the most probable kmer of the second sequence?
  - How can we solve this?

ttACCTtaac

gatgtctgtc

A:	1	0	0	0
C:	0	1	1	0
G:	0	0	0	0
T:	0	0	0	1

Pseudocounts

- Saying that something has a 0 probability of happening is not a good idea (especially in biology)
- We can add a count of 1 to each element of the matrix

Motifs

T	A	A	C
G	T	C	T
A	C	T	A
A	G	G	T

COUNT(*Motifs*)

A:	2+1	1+1	1+1	1+1
C:	0+1	1+1	1+1	1+1
G:	1+1	1+1	1+1	0+1
T:	1+1	1+1	1+1	2+1

PROFILE(*Motifs*)

3/8	2/8	2/8	2/8
1/8	2/8	2/8	2/8
2/8	2/8	2/8	1/8
2/8	2/8	2/8	3/8

GreedyMotifSearch with PseudoCounts (BA2E)

ttACCTtaac
gATGTctgtc
acgGCGTtag
ccctaACGAg
cgtcagAGGT

- Assuming we found the correct motifs in the first two sequences but the incorrect one on the third sequence:

ACCT
ATGT
acgG

A:	3+1	0+1	0+1	1+1	4/7	1/7	1/7	1/7
C:	0+1	2+1	1+1	0+1	1/7	3/7	2/7	1/7
G:	0+1	0+1	2+1	1+1	1/7	1/7	3/7	2/7
T:	0+1	1+1	0+1	2+1	1/7	2/7	1/7	3/7

- Apply to 4th sequence (ccctaACGAg):

ccct	ccta	ctaA	taAC	aACG	ACGA	CGAg
$18/7^4$	$3/7^4$	$2/7^4$	$1/7^4$	$16/7^4$	$36/7^4$	$2/7^4$

- The correct one is still selected!

Randomized algorithms

- Greedy algorithms can still get stuck in local optima
- Monte Carlo algorithms do not guarantee exact solution but are quick to find approximate solution, so they can be run multiple times

Randomized algorithms

- Considering MOTIFS(*Profile*, *Dna*) as a function that return the most probable k-mer of each sequence according to the pattern:

Profile

A:	4/5	0	0	1/5
C:	0	3/5	1/5	0
G:	1/5	1/5	4/5	0
T:	0	1/5	0	4/5

Dna

```
ttaccttaac
gatgtctgtc
acggcgttag
ccctaacgag
cgtcagaggt
```

MOTIFS(*Profile*, *Dna*)

tt**acct**taac
g**atgt**ctgtc
acg**gcgt**tag
cccta**acga**g
cgtcag**aggt**

- We can start from a random set of k-mers, generate a *Profile* from that, and compute MOTIFS(*Profile*, *Dna*)
- Hopefully the score of the new Motif will be better
- We can iterate this process multiple times

RandomizedMotifSearch (BA2F)

- Start with these random k-mers:
- Compute profile matrix
- Recalculate most probable k-mer of each string

ttACCT**taac**
gAT**GTct**gtc
ccgGCGTtag
c**acta**ACGAg
cgtcag**AGGT**

PROFILE(Motifs)

⬇

MOTIFS(PROFILE(Motifs),Dna)

⬇

PROFILE(MOTIFS(PROFILE(Motifs), Dna))

⬇

MOTIFS(PROFILE(MOTIFS(PROFILE(Motifs), Dna)), Dna)

RandomizedMotifSearch (BA2F)

```
RANDOMIZEDMOTIFSEARCH(Dna, k, t)
    randomly select k-mers Motifs = (Motif_1, ..., Motif_t) in each string from Dna
    BestMotifs ← Motifs
    while forever
        Profile ← PROFILE(Motifs)
        Motifs ← MOTIFS(Profile, Dna)
        if SCORE(Motifs) < SCORE(BestMotifs)
            BestMotifs ← Motifs
        else
            return BestMotifs
```

RandomizedMotifSearch

- What would happen if each nucleotide was equally probably at each position?

A:	0.25	0.25	0.25	0.25
C:	0.25	0.25	0.25	0.25
G:	0.25	0.25	0.25	0.25
T:	0.25	0.25	0.25	0.25

- Usually, that’s not the case, we get something already pointing to a solution, since the strings are not random

Gibbs Sampling

- The previous algorithm changes all sequence at the same time
- Might be better to change one at a time (why?)
- GibbsSampler selects one sequence at a time to replace its k-mer

ttacctt**aac**		t**tac**cttaac	ttacctt**aac**		ttacctt**aac**
g**ata**tctgtc		gat**atc**tgtc	g**ata**tctgtc		gatatc**tgt**c
acggcgttcg	→	acggcg**ttc**g	**acg**gcgttcg	→	**acg**gcgttcg
ccct**aaa**gag		ccctaa**aga**g	ccct**aaa**gag		ccct**aaa**gag
cgtc**aga**ggt		**cgt**cagaggt	cgtc**aga**ggt		cgtc**aga**ggt

RANDOMIZEDMOTIFSEARCH
(may change all *k*-mers in one step)

GIBBSSAMPLER
(changes one *k*-mer in one step)

Gibbs Sampling

- Both the sequence and the k-mer are randomly selected
  - Sequence is truly random
  - K-mer is according to its probability
  - We need to renormalize the k-mer probabilities
    - E.g. $(p_1, p_2, p_3) = (0.1, 0.2, 0.3)$

$$
\begin{aligned}
\textsc{Random}(0.1, 0.2, 0.3) &= \textsc{Random}(0.1/0.6, 0.2/0.6, 0.3/0.6) \\
&= \textsc{Random}(1/6, 1/3, 1/2)\,.
\end{aligned}
$$

- Run for a fixed number of iterations

Gibbs Sampling

ttACCT**taac**		ttACCT**taac**
gAT**GTct**gtc		gAT**GTct**gtc
ccgGCGTtag	$\longrightarrow$	----------
c**acta**ACGAg		c**acta**ACGAg
cgtcag**AGGT**		cgtcag**AGGT**

t	a	a	c
G	T	c	t
a	c	t	a
A	G	G	T

A:	3/8	2/8	2/8	2/8
C:	1/8	2/8	2/8	2/8
G:	2/8	2/8	2/8	1/8
T:	2/8	2/8	2/8	3/8

ccgG	cgGC	gGCG	GCGT	CGTt	GTta	Ttag
$4/8^4$	$8/8^4$	$8/8^4$	$24/8^4$	$12/8^4$	$16/8^4$	$8/8^4$

- We pick a random k-mer according to this prob. distribution
- Let's say we pick GCGT
- New we put that sequence again in the motif and choose randomly a new one

Gibbs Sampling

ttACCT**taac**		----------		G T c t
gAT**GTct**gtc		gAT**GTct**gtc		G C G T
ccg**GCGT**tag	$\longrightarrow$	ccg**GCGT**tag		a c t a
c**acta**ACGAg		c**acta**ACGAg		A G G T
cgtcag**AGGT**		cgtcag**AGGT**		

A:	3/8	1/8	1/8	2/8
C:	1/8	3/8	2/8	1/8
G:	3/8	2/8	3/8	1/8
T:	1/8	2/8	2/8	4/8

- We calculate the probabilities of all kmers of ttACCTtaac:

ttAC	tACC	ACCT	CCTt	CTta	Ttaa	taac
$2/8^4$	$2/8^4$	$72/8^4$	$24/8^4$	$8/8^4$	$4/8^4$	$1/8^4$

- The chosen kmers start to converge to the correct ones

GibbsSampler(BA2G)

```
GIBBSSAMPLER(Dna, k, t, N)
    randomly select k-mers Motifs = (Motif_1, ..., Motif_t) in each string from Dna
    BestMotifs ← Motifs
    for j ← 1 to N
        i ← RANDOM(t)
        Profile ← profile matrix formed from all strings in Motifs except for Motif_i
        Motif_i ← Profile-randomly generated k-mer in the i-th sequence
        if SCORE(Motifs) < SCORE(BestMotifs)
            BestMotifs ← Motifs
    return BestMotifs
```

Gibbs sampling

- May also get stuck in local optima
- But we can easily run it many time for many iterations
- Smaller changes will make it converge faster

Summary

- Greedy motif search
  - Iterates profile matrix according to current best k-mers
- Random search
  - Initial positions are randomly sampled
- Gibbs Sampling
  - At each step, remove a sequence and update the motif with weighted sampled match

Bibliography

- Compeau Chapter 2
- Rocha Chapter 11
  - Greedy Search is called Expectation Maximization here